專業排版與設計期中作業

LINE Messaging API 教學文件製作

座號： 5

班級： 資管系 4C

學號： 412630096

姓名： 利蓁琳

評分項目	配分	得分	評分說明
LaTeX 頁數達標	10%		文件達五頁以上，內容充實、有條理。
LaTeX 排版美觀與結構	15%		章節層次清晰，目錄可正常顯示。
LaTeX 進行 GitHub 版本控管	15%		文件成功上傳 GitHub，至少有三筆有實質變更的提交。
LaTeX 格式正確	20%		可編譯 PDF，段落、圖片與程式碼排版完善，行號自動產生。
LaTeX 內容完整性	20%		完整描述註冊過程與設定步驟，具技術深度。
LaTeX 程式碼品質與解說	20%		程式碼可運作、註解充分，並標註行號。

2026 年 11 月 13 日

章節目錄

摘要 **5**

1 Messaging API 簡介與架構 **6**

1.1 用途與基本名詞 6

1.2 Webhook 與回覆的關係 6

2 帳號註冊與 Channel 建立 **8**

2.1 建立官方帳號 8

2.2 啟用 Messaging API 8

2.3 確認設定位置與權限 9

3 憑證取得與環境設定 **10**

3.1 Channel secret 與 access token 10

3.2 PowerShell 環境變數 11

3.3 憑證保護 11

4 Webhook 設定與測試 **12**

4.1 建立公開 HTTPS 入口 12

4.2 Verify 與手機端驗收 12

4.3 為何驗證原始 body 12

5 完整範例程式與解說 **14**

5.1 設計與執行流程 14

5.2 接收與回覆程式 14

5.3 程式行號與逐段解說 18

5.3.1 依事件追蹤資料流 18

5.3.2 如何根據紀錄定位問題 19

5.4 錯誤處理與部署限制 19

6 測試、排錯與延伸應用 **20**

6.1 離線測試與線上測試的區別 20

6.2 線上實測與證據紀錄 21

6.3 常見問題排查 21

6.4 Push、Rich Menu 與 Flex Message 21

7 重現、版本管理與參考資料 **22**

7.1 檔案與編譯方式 22

7.2 Git 與 GitHub 交付 22

7.3 結論 22

7.4 官方參考資料 22

8 本文件 LaTeX 原始碼 **24**

圖目錄

1.1 回聲機器人的事件與回覆流程（自行繪製） 7

2.1 帳號開通與 Channel 建立路線圖（自行繪製） 8

3.1 三種憑證的用途與生命週期概念（自行繪製） 10

4.1 文字事件的同步處理時序（自行繪製；步驟 3 成功後才確認接收） 13

6.1 從程式邏輯到手機回覆的三層驗收（自行繪製） 20

表目錄

1.1 主要元件與責任 6

3.1 憑證用途與設定對照 10

5.1 app.py 行號與處理責任對照 18

6.1 驗收項目與預期結果 20

6.2 Webhook 常見錯誤與處理方式 21

程式目錄

3.1 設定環境變數並啟動服務 11

5.1 app.py：匯入模組與簽章驗證 14

5.2 app.py：建立回覆 API 請求 15

5.3 app.py：解析事件與分類錯誤 16

5.4 app.py：HTTP 接收端與回應 17

5.5 app.py：環境檢查與服務啟動 18

7.1 XeLaTeX 編譯命令 22

8.1 main.tex：本文件完整 LaTeX 原始碼 24

摘要

本文件以繁體中文說明 LINE Messaging API 的開通、憑證管理、Webhook 設定，以及接收文字後原文回覆的 Echo Bot。範例直接使用 Python 標準函式庫呼叫 REST API，讓讀者能清楚觀察 HTTP、JSON 與 HMAC 簽章之間的關係，不需安裝第三方 SDK。

閱讀順序依實際操作排列：先辦識帳號、Provider 與 Channel，再建立官方帳號並啟用 API，接著設定 secret、token 與 HTTPS 網址，最後啟動程式並執行測試。文件提供五張自行繪製的架構與操作圖、設定表、錯誤排查表，以及自動產生行號的完整程式碼。

實作範圍：本交付包含可執行程式與離線測試，不含使用者個人的 LINE 帳號、真實憑證、已部署服務或手機端成功截圖。文件中的驗收步驟供取得憑證後操作；不得將預期結果解讀為已完成線上驗證。班級為資管系 4C，座號為 5。

版本基準：註冊流程於 2026 年 11 月 13 日查閱 LINE 官方文件。服務畫面與方案可能改版，實際名稱以官方管理介面為準。參考資料集中於倒數第二章，最後一章列出本文件完整 LaTeX 原始碼。

第 1 章　Messaging API 簡介與架構

1.1　用途與基本名詞

Messaging API 是讓程式與 LINE 官方帳號互動的介面。當使用者加入好友、傳送訊息或操作互動元件時，LINE 平台可將事件送至指定伺服器；伺服器依商業邏輯呼叫 API 回覆。它適合校園問答、預約通知、訂單進度查詢與活動資訊服務。

LINE 帳號供使用者登入；Business ID 用於官方帳號相關服務；官方帳號是使用者加入好友並聊天的對象。Provider 表示服務提供者，Channel 則是該服務使用 LINE 平台功能的設定單位。開發者須管理 Channel 的權限、密鑰與端點。

表 1.1: 主要元件與責任

元件	本範例的責任
使用者	加入好友並傳送文字
LINE 平台	產生事件、附上簽章並傳送 Webhook
HTTPS 通道	將公開網址導向本機 8000 埠
Python 伺服器	驗證簽章、解析事件、發送回覆請求
Messaging API	檢查 access token 並將回覆送回聊天室

1.2　Webhook 與回覆的關係

Webhook 是由 LINE 主動發起的 HTTP POST；回覆 API 則由機器人主動向 LINE 發起 HTTPS 請求。兩者是不同方向的連線。將 Webhook 回傳為 200 僅表示事件已被接收，不會自動產生聊天訊息，程式仍須呼叫回覆端點。

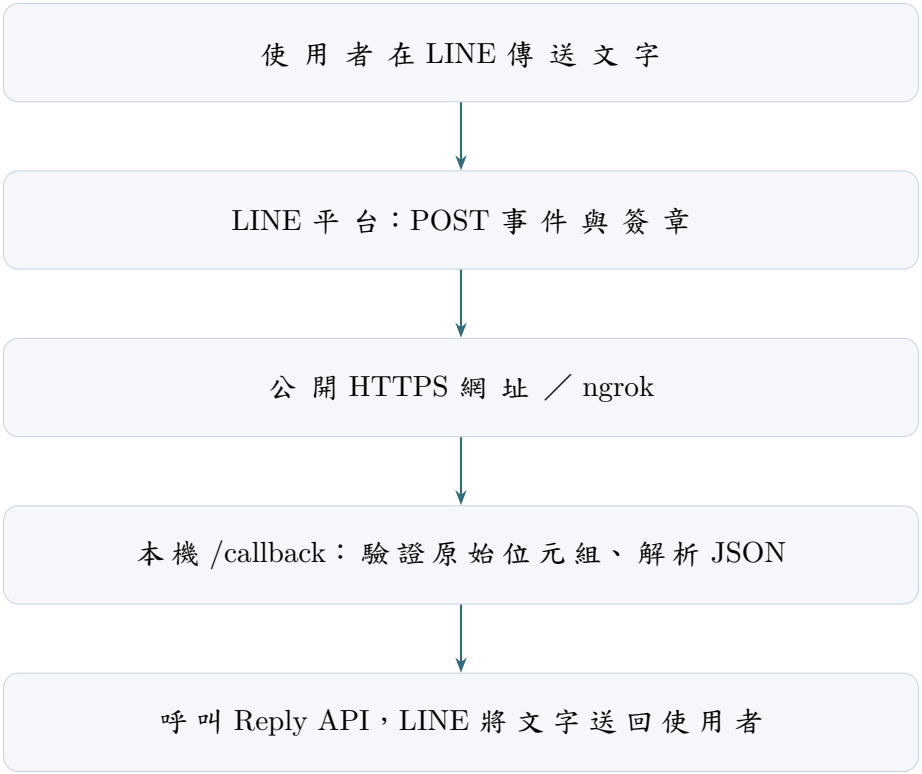


圖 1.1: 回聲機器人的事件與回覆流程（自行繪製）

第 2 章　帳號註冊與 Channel 建立

2.1　建立官方帳號

1. 開啟 `https://manager.line.biz/`，依畫面登入或註冊 Business ID。可選擇自己的 LINE 帳號或電子郵件方式；完成必要驗證。
2. 建立新的 LINE 官方帳號，填寫帳號名稱、聯絡資料及介面要求的必要欄位。名稱可使用課程實驗用途，避免冒用品牌。
3. 完成後在 Official Account Manager 確認帳號存在，並記錄其顯示名稱與識別資訊。先確認登入身分具有管理權限，才能繼續設定。

2.2　啟用 Messaging API

1. 在官方帳號管理介面選取剛建立的帳號，開啟「設定」中的「Messaging API」，啟用此功能。此動作會建立對應的 Messaging API Channel。
2. 若首次使用開發者功能，依畫面登錄開發者姓名與電子郵件。這是開發者資料，不是機器人的公開顯示名稱。
3. 選擇或建立 Provider。Provider 應對應實際提供服務的個人或組織；若有 LINE Login 等整合需求，先確認是否需使用同一 Provider。
4. 仔細核對 Provider 再確認啟用。官方文件指出，官方帳號指派 Provider 後無法更換或解除，不能把這一步當成可隨意修改的暫存設定。
5. 登入 `https://developers.line.biz/console/`，選取該 Provider，確認其下出現 Messaging API Channel。

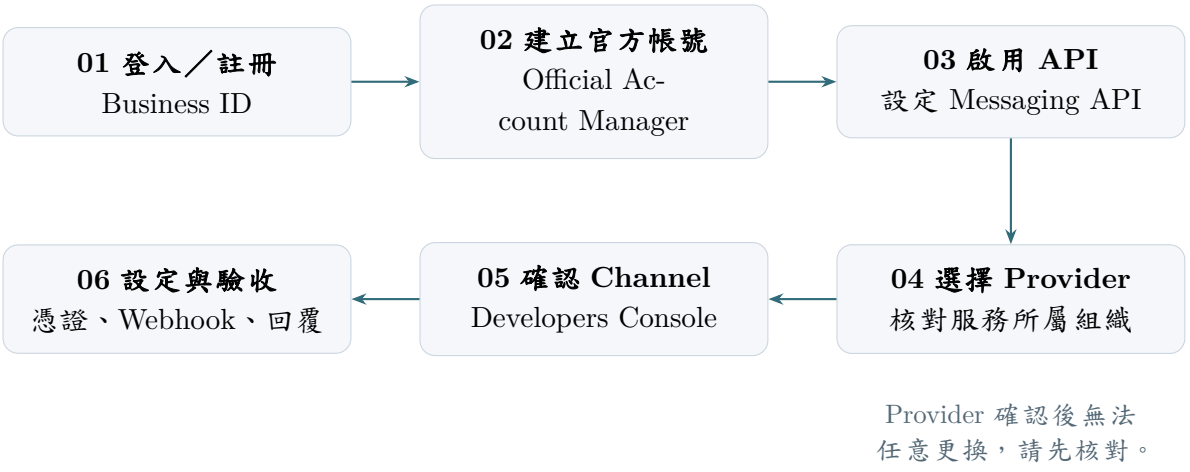


圖 2.1: 帳號開通與 Channel 建立路線圖（自行繪製）

2.3 確認設定位置與權限

在 Channel 的 Basic settings 檢查 Channel ID、名稱與 Channel secret；在 Messaging API 分頁設定 token、Webhook 及加入好友資訊。Channel ID 是識別碼，不能代替 secret 或 token。

如果找不到 Channel，先確認兩個管理平台是否使用相同登入身分、Provider 是否正確，以及帳號是否有管理權限。不要因為介面不同而重複建立多個官方帳號。本文採用官方帳號端啟用 API 的流程，避免沿用過時的直接新增 Messaging API Channel 教學。

第 3 章　憑證取得與環境設定

3.1　Channel secret 與 access token

Channel secret 是驗證 Webhook 的共享密鑰。進入 Channel 的 Basic settings，取得 Channel secret，保存在伺服器環境變數。收到請求後，使用它計算原始 body 的 HMAC-SHA256，再進行 Base64 編碼，與 X-Line-Signature 比對。

Channel access token 用於授權伺服器呼叫 Messaging API，放在 HTTP Authorization 標頭，格式為 Bearer 加上 token。教學環境可在 Messaging API 分頁的長效 token 區塊發行 token；若已有 token，勿為了複製而反覆重發。正式服務宜評估官方建議的可指定效期 token v2.1 與輪替策略。

表 3.1: 憑證用途與設定對照

資料	取得位置	用途
Channel ID	Basic settings	識別頻道，不可作驗證密鑰
Channel secret	Basic settings	驗證 Webhook 簽章
Access token	Messaging API ／ token 發行流程	授權回覆 API
Reply token	Webhook 事件	單次回覆特定事件

Channel secret ｜ 確認事件來源
LINE 平台與接收端共享，用於 HMAC-SHA256 簽章驗證。

Access token ｜ 授權呼叫 API
機器人伺服器持有，放在 Authorization: Bearer 標頭。

Reply token ｜ 回覆這一次事件
從 Webhook 取得，僅供單次回覆，不能當成長效憑證。

三者責任不同；不得互相替代，也不要放入公開截圖與版本紀錄。

圖 3.1: 三種憑證的用途與生命週期概念（自行繪製）

3.2 PowerShell 環境變數

使用 Python 3.10 以上版本。範例僅用標準函式庫，無須安裝 Flask 或 LINE SDK。請在啟動程式的同一個終端機設定以下變數；占位文字須替換為自己的真值。

程式碼 3.1: 設定環境變數並啟動服務

```
$env:LINE_CHANNEL_SECRET = 'YOUR_CHANNEL_SECRET'
$env:LINE_CHANNEL_ACCESS_TOKEN = 'YOUR_CHANNEL_ACCESS_TOKEN'
python app.py
```

環境變數僅對目前終端機及其子程序生效。程式不會自動讀取.env；隨附.env.example 只說明欄位。缺少變數時程式會立即退出，避免誤以為空憑證也能完成回覆。

3.3 憑證保護

不要把真實憑證放入程式碼、報告、截圖或 Git commit。若曾外洩，應在平台撤銷／重發，更新服務環境後再測試；只刪除檔案無法移除既有 Git 歷史中的秘密。程式紀錄只保留必要錯誤訊息，不輸出聊天內容、Authorization 或 reply token。

第 4 章　Webhook 設定與測試

4.1　建立公開 HTTPS 入口

本機程式僅監聽 127.0.0.1:8000。LINE 無法連到個人電腦的 localhost，因此開發時需建立公開 HTTPS 通道，正式服務則部署至具有有效憑證的伺服器。

1. 先執行程式碼 3.1，確認看到 Listening 訊息。
2. 從 `https://ngrok.com/docs/` 安裝 ngrok，登入自己的帳號並依官方指引設定 authtoken。不要把 ngrok 的 token 誤當 LINE token。
3. 另開終端機執行 `ngrok http 8000`，保持兩個程序持續運作。
4. 複製 ngrok 顯示的 HTTPS forwarding 網址，後面加上 `/callback`。例如 `https://example.ngrok-free.app/callback` 是格式示意，並非可用端點。
5. 開啟 Developers Console 的 Channel，於 Messaging API 分頁編輯 Webhook URL，貼上完整網址並儲存。

4.2　Verify 與手機端驗收

點選 Verify 測試連通性，再啟用 Use webhook。Verify 可能傳送 events 為空陣列的請求；程式必須通過簽章驗證並回覆 HTTP 200，不應因缺少訊息就拋出例外。

接著透過 Channel 顯示的 QR code 將官方帳號加入好友。依管理介面的回應設定關閉會與程式重複回覆的自動回應；視需求停用加入好友歡迎訊息。傳送「你好 LINE」，預期收到完全相同的文字。再傳送圖片，範例會略過並回覆 Webhook 200，不會產生文字回聲。

兩種成功必須分開確認：Verify 成功表示端點可接收驗證請求；手機實際收到回覆才證明事件、token 與 Reply API 全部連通。本文未代替讀者登入個人帳號，這兩項須由本人完成。

4.3　為何驗證原始 body

簽章以傳輸時的位元組計算，空白、換行或 Unicode 編碼的差異都會改變結果。因此必須先讀取原始 bytes 並驗證，再呼叫 JSON 解析。不能先解析 JSON、重新序列化後才驗證。範例使用固定時間比較函式降低比對過程的資訊洩漏。

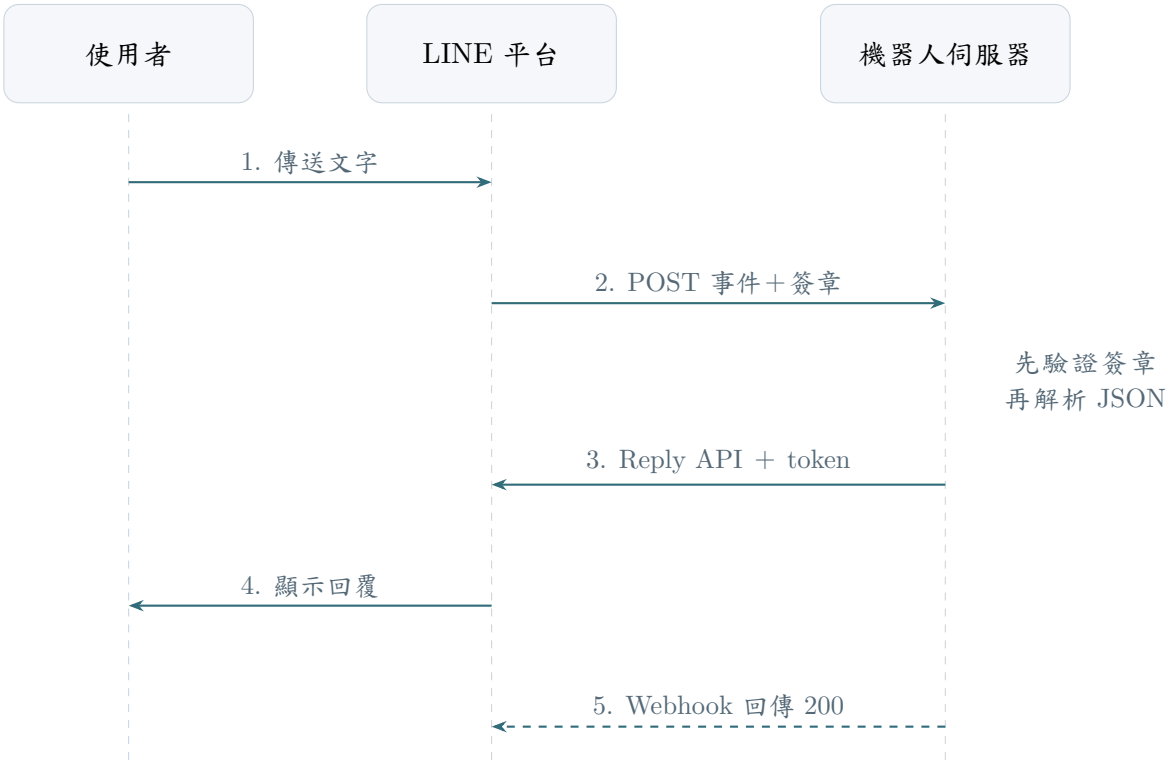


圖 4.1: 文字事件的同步處理時序（自行繪製；步驟 3 成功後才確認接收）

第 5 章　完整範例程式與解說

5.1　設計與執行流程

本實作的接收端與 API 呼叫均使用 Python 標準函式庫。圖 1.1 對應下列函式：verify_signature 保護事件來源；reply_text 建立授權請求；process_webhook 決定哪些事件需要回覆；make_handler 將 HTTP POST 轉交核心處理函式。

每個文字事件使用自己的 replyToken。回覆 token 只能使用一次，應收到後立即使用；官方規範要求在收到 Webhook 後一分鐘內使用，重送事件另有條件，不應將 token 當成可長期保存的憑證。程式不盲目重試 API，以避免重複使用 token。

非文字事件、standby 模式或沒有可用 reply token 的事件會略過。驗證成功但 events 為空則直接回覆 200。API 網路失敗回傳 502，部署時可據此觀測錯誤。此同步教學實作未加入事件去重、佇列與資料庫，不應直接當成高流量正式服務。

5.2　接收與回覆程式

下列程式直接從交付的 app.py 匯入，左側行號由 listings 自動產生。註解採英文以方便程式碼跨環境顯示，本節以繁體中文補足解說。程式依處理責任分成五段，行號連續對應同一份 app.py，可直接執行交付的完整檔案。

程式碼 5.1: app.py：匯入模組與簽章驗證

```
1  """LINE echo bot. Python 3.10+, standard library only; local demo server."""
2  import base64
3  import hashlib
4  import hmac
5  import json
6  import logging
7  import os
8  from http.server import BaseHTTPRequestHandler, HTTPServer
9  from urllib.error import HTTPError, URLError
10 from urllib.request import Request, urlopen
11
12 REPLY_URL = "https://api.line.me/v2/bot/message/reply"
13 LOG = logging.getLogger(__name__)
14
15
16 def verify_signature(body, signature, secret):
17     """Verify original bytes BEFORE parsing JSON; never reserialize first."""
18     digest = hmac.new(secret.encode("utf-8"), body, hashlib.sha256).digest()
19     expected = base64.b64encode(digest)
20     return hmac.compare_digest(expected, signature.encode("utf-8"))
```

程式碼 5.2: app.py：建立回覆 API 請求

```
23 def reply_text(token, reply_token, text):
24     """Reply tokens are single-use. Do not blindly retry this POST."""
25     payload = {"replyToken": reply_token,
26                "messages": [{"type": "text", "text": text}]}
27     data = json.dumps(payload, ensure_ascii=False).encode("utf-8")
28     req = Request(REPLY_URL, data=data, method="POST", headers={
29         "Authorization": "Bearer " + token,
30         "Content-Type": "application/json"})
31     with urlopen(req, timeout=10) as response:
32         response.read()
```

程式碼 5.3: app.py：解析事件與分類錯誤

```
def process_webhook(body, signature, secret, token, sender=reply_text):
    """Return an HTTP status; sender is injectable for offline tests."""
    if not verify_signature(body, signature, secret):
        return 400
    try:
        payload = json.loads(body)
    except (ValueError, UnicodeDecodeError):
        return 400
    if not isinstance(payload, dict):
        return 400
    events = payload.get("events")
    if not isinstance(events, list):
        return 400
    # Verification sends an empty events array: acknowledge without replying.
    for event in events:
        if not isinstance(event, dict):
            return 400
        message = event.get("message", {})
        if (event.get("type") != "message"
                or not isinstance(message, dict)
                or message.get("type") != "text"):
            continue
        if event.get("mode") == "standby":
            continue
        reply_token = event.get("replyToken")
        text = message.get("text")
        if not isinstance(reply_token, str) or not reply_token:
            continue
        if not isinstance(text, str) or not text:
            continue
        try:
            sender(token, reply_token, text)
        except HTTPError as exc:
            # Log status only: response bodies can contain private data.
            LOG.error("LINE reply HTTP status=%s", exc.code)
            return 502
        except (URLError, TimeoutError, OSError):
            # Never log exception text, user messages, or credentials.
            LOG.error("LINE reply connection failed")
            return 502
    return 200
```

程式碼 5.4: app.py：HTTP 接收端與回應

```
78  def make_handler(secret, token):
79      class Handler(BaseHTTPRequestHandler):
80          def do_POST(self):
81              if self.path != "/callback":
82                  self.send_error(404)
83                  return
84              try:
85                  size = int(self.headers.get("Content-Length", "0"))
86              except ValueError:
87                  self.send_error(400)
88                  return
89              if not 0 < size <= 1024 * 1024:
90                  self.send_error(413)
91                  return
92              self.connection.settimeout(15)
93              try:
94                  body = self.rfile.read(size)
95              except (TimeoutError, OSError):
96                  self.send_error(408)
97                  return
98              if len(body) != size:
99                  self.send_error(400)
100                 return
101             signature = self.headers.get("X-Line-Signature", "")
102             status = process_webhook(body, signature, secret, token)
103             response = b"OK" if status == 200 else b"Rejected"
104             self.send_response(status)
105             self.send_header("Content-Type", "text/plain; charset=utf-8")
106             self.send_header("Content-Length", str(len(response)))
107             self.end_headers()
108             self.wfile.write(response)
109     return Handler
```

程式碼 5.5: app.py：環境檢查與服務啟動

```
112 def main():
113     secret = os.environ.get("LINE_CHANNEL_SECRET", "").strip()
114     token = os.environ.get("LINE_CHANNEL_ACCESS_TOKEN", "").strip()
115     if not secret or not token:
116         raise SystemExit("Set LINE_CHANNEL_SECRET and LINE_CHANNEL_ACCESS_TOKEN")
117     # HTTP is local only; ngrok terminates public HTTPS during development.
118     server = HTTPServer(("127.0.0.1", 8000), make_handler(secret, token))
119     print("Listening on http://127.0.0.1:8000/callback")
120     try:
121         server.serve_forever()
122     except KeyboardInterrupt:
123         pass
124     finally:
125         server.server_close()
126
127
128 if __name__ == "__main__":
129     main()
```

5.3 程式行號與逐段解說

表中行號與程式碼 5.1 至 5.5 對照。可先閱讀核心事件處理，再追蹤驗證與回覆函式，最後確認 HTTP 接收端如何串接。

表 5.1: app.py 行號與處理責任對照

行號	功能	說明
16–22	簽章驗證	使用 secret 對原始 bytes 計算 HMAC-SHA256，再以固定時間比較驗證 Base64 簽章；此時尚未解析 JSON。
23–34	發送回覆	將文字與 replyToken 編為 JSON，於 Authorization 加入 access token，設定十秒逾時並呼叫 Reply API。
35–77	事件處理	先驗證再解析，略過非文字及 standby 事件；空事件回覆 200，API 錯誤回覆 502。
78–111	HTTP 接收	限制路徑、內容長度及讀取時間，將原始 body 與標頭交給核心函式，再明確回傳 HTTP 狀態。
112–129	啟動與關閉	檢查環境變數後只監聽本機，配合 HTTPS 通道；按 Ctrl+C 時釋放伺服器資源。

5.3.1 依事件追蹤資料流

事件進入核心函式後，先驗證來源，再解析事件陣列。每筆事件必須是文字訊息且具有 replyToken，才將文字交給 reply_text。收到空陣列時迴圈不執行，仍回傳 200，因此能接受平台的驗證請求。

5.3.2 如何根據紀錄定位問題

API 回傳 HTTP 錯誤時，僅記錄狀態碼。例如 401 可優先檢查 access token，400 可檢查請求參數及 replyToken；連線失敗則記錄 connection failed。接收端皆回傳 502 表示上游回覆未完成，詳細分類應查看伺服器紀錄。程式不輸出 API 回應 body 或例外原文，以避免敏感資訊進入紀錄。

5.4 錯誤處理與部署限制

請求路徑限定為 /callback，輸入大小上限為 1 MiB；HTTP body 讀取與對外 API 皆設逾時。錯誤簽章及錯誤資料格式回傳 400。憑證缺漏時於啟動階段中止，避免服務啟動後才發現設定錯誤。

本範例的 HTTPServer 是單一程序的教學伺服器。正式環境應改用可靠的應用框架與伺服器、HTTPS 反向代理、祕密管理與可觀測性機制。若啟用 Webhook redelivery，須以 webhookEventId 設計持久化去重與工作狀態管理；單純把已處理 ID 存入記憶體無法承受重啟，多事件請求部分成功時也可能重複處理。須先完成這些機制再依需求啟用重送。

第 6 章　測試、排錯與延伸應用

6.1　離線測試與線上測試的區別

執行 `python -m unittest -v test_app.py`，測試以假 secret 計算簽章，以 mock 取代對外傳訊，不需 LINE 憑證。涵蓋正常文字回覆、內容遭竄改、缺少簽章、空事件、非文字與 standby 事件、錯誤 JSON 結構、無效 JSON、網路錯誤及輸出請求格式。另以本機 HTTPServer 與實際 HTTP POST 驗證路徑錯誤、空 body、大小限制、無效長度、無效簽章、有效驗證請求與上游失敗，並確認錯誤紀錄只含 HTTP 狀態碼。共 17 項測試，均不使用真實憑證。

離線通過表示核心邏輯符合這些測試案例，仍須以真實帳號、有效憑證、公開 HTTPS 與手機訊息完成端對端驗收。不得以 mock 成功取代真實串接證據。

表 6.1: 驗收項目與預期結果

操作	預期結果
缺少環境變數啟動	顯示缺少設定並退出
官方 Verify	回覆 200，介面顯示成功
手機傳送文字	收到相同文字一次
傳送圖片	不產生文字回聲
錯誤簽章	400，不呼叫回覆 API
無效 access token	API 失敗，接收端回覆 502



圖 6.1: 從程式邏輯到手機回覆的三層驗收（自行繪製）

6.2 線上實測與證據紀錄

以下項目需由本人登入 LINE 平台並實際操作，尚未執行。完成後可補入去除敏感資訊的截圖，並記錄時間、操作與觀察結果。

1. Channel 與 Webhook 設定：擷取 API 已啟用及 Use webhook 設定畫面，遮蔽 secret、token 與不需公開的識別資料。
2. Verify 成功：確認網址含 /callback 且顯示成功，記錄測試時間。
3. 手機端文字回覆：傳送一則文字，截取使用者輸入與相同回覆，確認未發生重複回覆。
4. 非文字測試：傳送圖片，確認伺服器正常接收且未產生文字回聲。

上述是待執行的驗收程序；隨附測試紀錄只證明離線與本機 HTTP 測試結果。

6.3 常見問題排查

表 6.2: Webhook 常見錯誤與處理方式

現象	可能原因	處理方式
Verify 連線失敗	本機或 ngrok 未執行	確認兩個終端機持續運作
404	忘記 /callback 或路徑拼錯	修正完整 Webhook URL
400	secret 錯誤、簽章缺漏、body 被改寫	核對 Channel 並保留原始 bytes
502	token 過期或錯誤、網路失敗、reply token 無效	查平台狀態與憑證，不盲目重試
Verify 成功但無訊息	Use webhook 未開、未加好友	啟用設定並用手機實測
收到兩次不同回覆	平台自動回應仍啟用	檢查官方帳號回應設定
隔天無法連線	通道網址更換	更新 Console 的 HTTPS 網址

6.4 Push、Rich Menu 與 Flex Message

Reply 用於回覆事件；Push 可由服務主動發送通知，但仍須遵守可發送對象、方案與用量限制。預約提醒可採 Push，切勿在沒有合法用途與適當告知的情況下大量發送。

Rich Menu 在聊天畫面提供固定入口，適合查詢、預約與客服。Flex Message 以 JSON 描述卡片等介面，適合訂單或活動資訊；可先用官方模擬工具預覽，並提供有意義的替代文字。這些功能不改變接收 Webhook 時必須驗證簽章的原則。

第 7 章　重現、版本管理與參考資料

7.1　檔案與編譯方式

學號資料夾包含 main.tex、app.py、test_app.py、.env.example、README.md、編譯後 PDF 與驗證紀錄。Windows 安裝 XeLaTeX 後，在此資料夾執行下列指令三次，使頁碼與四種目錄穩定。

程式碼 7.1: XeLaTeX 編譯命令

```
xelatex -interaction=nonstopmode -halt-on-error -jobname=412630096 main.tex
```

本文件使用 xeCJK 與標楷體（DFKai-SB）顯示繁體中文，listings 產生程式行號，TikZ 繪製架構圖，hyperref 建立目錄與交叉引用。若更換作業系統，請將字型名稱改為已安裝且可顯示繁體中文的字型後重新編譯。

7.2　Git 與 GitHub 交付

本機以功能分段提交：先建立文件架構與執行說明，再加入機器人與測試，最後完成教學排版及驗證。這些提交包含實際內容變更，不使用空提交湊數。本作業指定 GitHub 儲存庫為 Charlotte1107/professional-layout-design。完成編譯與驗證後，將本機提交推送至該儲存庫，並核對遠端分支與本機版本一致。

version-history.txt 記錄實際本機提交，repository.bundle 保存可還原的版本歷史。ZIP 不包含真實憑證；.gitignore 排除.env 與編譯暫存檔。雲端版本可於指定儲存庫查閱；實際推送與檔案完整性結果另記錄於交付驗證紀錄。

7.3　結論

完整串接須同時完成官方帳號啟用、Channel 設定、秘密管理、HTTPS 端點、簽章驗證與訊息回覆。最容易誤判的部分是把 Verify 成功視為所有功能完成；透過離線核心測試與手機端驗收分層檢查，才能定位問題並確認真正的使用者體驗。

7.4　官方參考資料

以下資料查閱日期均為 2026 年 11 月 13 日。本文步驟以自行整理方式呈現，架構圖為自行繪製，未使用虛構的操作截圖。

1. LINE Developers，Get started with the Messaging API：

https://developers.line.biz/en/docs/messaging-api/getting-started/。

2. LINE Developers，Build a bot：

   https://developers.line.biz/en/docs/messaging-api/building-bot/。

3. LINE Developers，Verify webhook signature：

   https://developers.line.biz/en/docs/messaging-api/verify-webhook-signature/。

4. LINE Developers，Messaging API reference（Reply message、Webhook event）：

   https://developers.line.biz/en/reference/messaging-api/。

5. ngrok，官方使用說明：https://ngrok.com/docs/。

第 8 章　本文件 LaTeX 原始碼

本章使用 lstinputlisting 直接載入 main.tex，讓實際排版來源可供查核。載入的是文字內容，不會遞迴執行其中指令；標題會自動加入程式目錄。修改本文後重新編譯，即可同步更新原始碼清單。

程式碼 8.1: main.tex：本文件完整 LaTeX 原始碼

```
\documentclass[12pt,a4paper]{report}
\usepackage{fontspec}
\usepackage{xeCJK}
\setCJKmainfont[AutoFakeBold=2]{DFKai-SB}
\setCJKsansfont[AutoFakeBold=2]{DFKai-SB}
\setCJKmonofont{DFKai-SB}
\usepackage[top=22mm,bottom=22mm,left=22mm,right=22mm]{geometry}
\usepackage{graphicx,xcolor,booktabs,longtable,array}
\usepackage{tikz}
\usetikzlibrary{arrows.meta,positioning,shapes.geometric,calc}
\usepackage{listings}
\usepackage{caption}
\usepackage{needspace}

\usepackage{xurl}
\usepackage[unicode,hidelinks]{hyperref}
\definecolor{linegreen}{HTML}{07883F}
\definecolor{codebg}{HTML}{F5F7FA}
\definecolor{ink}{HTML}{24384B}
\definecolor{accent}{HTML}{326C7A}
\definecolor{muted}{HTML}{546876}
\definecolor{border}{HTML}{CBD5DF}
\definecolor{stringink}{HTML}{854B49}
\tikzset{card/.style={draw=border,fill=codebg,rounded corners=2mm,
 align=center,font=\small,minimum height=13mm,inner sep=3mm},
 flow/.style={-{Stealth[length=2mm]},draw=accent,line width=.8pt},
 note/.style={font=\footnotesize,align=center,text=muted}}
\captionsetup{font=small,labelfont=bf}
\captionsetup[lstlisting]{singlelinecheck=false,skip=9pt}
\renewcommand{\contentsname}{章節目錄}
\renewcommand{\listfigurename}{圖目錄}
\renewcommand{\listtablename}{表目錄}
\renewcommand{\lstlistlistingname}{程式目錄}
\renewcommand{\lstlistingname}{程式碼}
\renewcommand{\figurename}{圖}
\renewcommand{\tablename}{表}
\renewcommand{\chaptername}{章}
\pagestyle{plain}
\setlength{\parindent}{2em}
```

```
\setlength{\parskip}{0.3em}
\linespread{1.12}
\setlength{\emergencystretch}{1em}
% Match the supplied exam: inline chapter number/title, black text, plain footer.
\makeatletter
\def\@makechapterhead#1{\vspace*{0pt}{\parindent\z@\raggedright
 \normalfont\Large\bfseries 第\thechapter 章\quad #1\par\nobreak\vskip 22pt}}
\def\@makeschapterhead#1{\vspace*{0pt}{\parindent\z@\raggedright
 \normalfont\Large\bfseries #1\par\nobreak\vskip 18pt}}
\makeatother
\lstset{basicstyle=\ttfamily\fontsize{9.5}{12.5}\selectfont\color{ink},
 numbers=left,numberstyle=\ttfamily\fontsize{7}{9}\selectfont\color{muted},
 numbersep=10pt,stepnumber=1,breaklines=true,breakatwhitespace=false,
 columns=fullflexible,keepspaces=true,showstringspaces=false,
 frame=single,frameround=tttt,framerule=.4pt,rulecolor=\color{border},
 framesep=7pt,xleftmargin=25pt,xrightmargin=8pt,framexleftmargin=20pt,
 backgroundcolor=\color{codebg},keywordstyle=\bfseries\color{accent},
 commentstyle=\normalfont\ttfamily\color{muted},stringstyle=\color{stringink},
 captionpos=t,tabsize=4,aboveskip=16pt,belowskip=16pt,
 abovecaptionskip=10pt,belowcaptionskip=8pt}
\hypersetup{pdftitle={LINE Messaging API 註冊申請與實作},pdfauthor={利蓁琳}}
\begin{document}
\begin{titlepage}
\centering
\vspace*{17mm}
{\LARGE\bfseries 專業排版與設計 期中作業\par}
\vspace{15mm}
{\LARGE LINE Messaging API 教學文件製作\par}
\vspace{15mm}
\renewcommand{\arraystretch}{1.6}
\begin{tabular}{rl}
座號：&\makebox[44mm][l]{5}\\
班級：&\makebox[44mm][l]{資管系4C}\\
學號：&\makebox[44mm][l]{412630096}\\
姓名：&\makebox[44mm][l]{利蓁琳}
\end{tabular}
\vspace{16mm}
\par\noindent
{\small\renewcommand{\arraystretch}{1.35}
\begin{tabular}{|p{44mm}|p{10mm}|p{10mm}|p{77mm}|}\hline
評分項目 & 配分 & 得分 & 評分說明\\\hline
\LaTeX 頁數達標 & 10\% & & 文件達五頁以上，內容充實、有條理。\\\hline
\LaTeX 排版美觀與結構 & 15\% & & 章節層次清晰，目錄可正常顯示。\\\hline
\LaTeX 進行 GitHub 版本控管 & 15\% & & 文件成功上傳 GitHub，至少有三筆有實質變更的提交。
    \\\hline
\LaTeX 格式正確 & 20\% & & 可編譯 PDF，段落、圖片與程式碼排版完善，行號自動產生。\\\hline
\LaTeX 內容完整性 & 20\% & & 完整描述註冊過程與設定步驟，具技術深度。\\\hline
\LaTeX 程式碼品質與解說 & 20\% & & 程式碼可運作、註解充分，並標註行號。\\\hline
\end{tabular}}
\vfill
{\large 2026 年 11 月 13 日\par}
```

```
\end{titlepage}
\setcounter{page}{2}
\tableofcontents
\clearpage
\begingroup
\let\clearpage\relax
\listoffigures
\vspace{12mm}
\listoftables
\vspace{12mm}
\lstlistoflistings
\endgroup
\clearpage
\chapter*{摘要}
\addcontentsline{toc}{chapter}{摘要}
本文件以繁體中文說明 LINE Messaging API 的開通、憑證管理、Webhook 設定，以及接收文字後原文回覆的 Echo Bot。範例直接使用 Python 標準函式庫呼叫 REST API，讓讀者能清楚觀察 HTTP、JSON 與 HMAC 簽章之間的關係，不需安裝第三方 SDK。

閱讀順序依實際操作排列：先辨識帳號、Provider 與 Channel，再建立官方帳號並啟用 API，接著設定 secret、token 與 HTTPS 網址，最後啟動程式並執行測試。文件提供五張自行繪製的架構與操作圖、設定表、錯誤排查表，以及自動產生行號的完整程式碼。

\textbf{實作範圍：}本交付包含可執行程式與離線測試，不含使用者個人的 LINE 帳號、真實憑證、已部署服務或手機端成功截圖。文件中的驗收步驟供取得憑證後操作；不得將預期結果解讀為已完成線上驗證。班級為資管系4C，座號為5。

\textbf{版本基準：}註冊流程於 2026 年 11 月 13 日查閱 LINE 官方文件。服務畫面與方案可能改版，實際名稱以官方管理介面為準。參考資料集中於倒數第二章，最後一章列出本文件完整 LaTeX 原始碼。
\clearpage
\chapter{Messaging API 簡介與架構}
\section{用途與基本名詞}
Messaging API 是讓程式與 LINE 官方帳號互動的介面。當使用者加入好友、傳送訊息或操作互動元件時，LINE 平台可將事件送至指定伺服器；伺服器依商業邏輯呼叫 API 回覆。它適合校園問答、預約通知、訂單進度查詢與活動資訊服務。

LINE 帳號供使用者登入；Business ID 用於官方帳號相關服務；官方帳號是使用者加入好友並聊天的對象。Provider 表示服務提供者，Channel 則是該服務使用 LINE 平台功能的設定單位。開發者須管理 Channel 的權限、密鑰與端點。
\begin{table}[htbp]
\centering\caption{主要元件與責任}\label{tab:components}
\begin{tabular}{p{32mm}p{100mm}}\toprule
元件 & 本範例的責任\\\midrule
使用者 & 加入好友並傳送文字\\
LINE 平台 & 產生事件、附上簽章並傳送 Webhook\\
HTTPS 通道 & 將公開網址導向本機 8000 埠\\
Python 伺服器 & 驗證簽章、解析事件、發送回覆請求\\
Messaging API & 檢查 access token 並將回覆送回聊天室\\\bottomrule
\end{tabular}
\end{table}
```

```
\section{Webhook 與回覆的關係}
Webhook 是由 LINE 主動發起的 HTTP POST；回覆 API 則由機器人主動向 LINE 發起 HTTPS 請求。兩者是不同方向的連線。將 Webhook 回傳為 200 僅表示事件已被接收，不會自動產生聊天訊息，程式仍須呼叫回覆端點。
\begin{figure}[htbp]
\centering
\begin{tikzpicture}[node distance=9mm,box/.style={card,text width=115mm},>=Stealth]
\node[box] (u) {使用者在 LINE 傳送文字};
\node[box,below=of u] (l) {LINE 平台：POST 事件與簽章};
\node[box,below=of l] (t) {公開 HTTPS 網址／ngrok};
\node[box,below=of t] (s) {本機 /callback：驗證原始位元組、解析 JSON};
\node[box,below=of s] (r) {呼叫 Reply API，LINE 將文字送回使用者};
\draw[flow] (u)--(l);\draw[flow] (l)--(t);\draw[flow] (t)--(s);\draw[flow] (s)--(r);
\end{tikzpicture}
\caption{回聲機器人的事件與回覆流程（自行繪製）}\label{fig:flow}
\end{figure}
\clearpage
\chapter{帳號註冊與 Channel 建立}
\section{建立官方帳號}
\begin{enumerate}
\item 開啟 \url{https://manager.line.biz/}，依畫面登入或註冊 Business ID。可選擇自己的 LINE 帳號或電子郵件方式；完成必要驗證。
\item 建立新的 LINE 官方帳號，填寫帳號名稱、聯絡資料及介面要求的必要欄位。名稱可使用課程實驗用途，避免冒用品牌。
\item 完成後在 Official Account Manager 確認帳號存在，並記錄其顯示名稱與識別資訊。先確認登入身分具有管理權限，才能繼續設定。
\end{enumerate}
\section{啟用 Messaging API}
\begin{enumerate}
\item 在官方帳號管理介面選取剛建立的帳號，開啟「設定」中的「Messaging API」，啟用此功能。此動作會建立對應的 Messaging API Channel。
\item 若首次使用開發者功能，依畫面登錄開發者姓名與電子郵件。這是開發者資料，不是機器人的公開顯示名稱。
\item 選擇或建立 Provider。Provider 應對應實際提供服務的個人或組織；若有 LINE Login 等整合需求，先確認是否需使用同一 Provider。
\item 仔細核對 Provider 再確認啟用。官方文件指出，官方帳號指派 Provider 後無法更換或解除，不能把這一步當成可隨意修改的暫存設定。
\item 登入 \url{https://developers.line.biz/console/}，選取該 Provider，確認其下出現 Messaging API Channel。
\end{enumerate}

\begin{figure}[htbp]
\centering
\begin{tikzpicture}[node distance=9mm and 9mm]
\node[card,text width=40mm] (a) {\textbf{01 登入／註冊}\\Business ID};
\node[card,text width=40mm,right=of a] (b) {\textbf{02 建立官方帳號}\\Official Account Manager};
\node[card,text width=40mm,right=of b] (c) {\textbf{03 啟用 API}\\設定 Messaging API};
\node[card,text width=40mm,below=14mm of c] (d) {\textbf{04 選擇 Provider}\\核對服務所屬組織};
\node[card,text width=40mm,left=of d] (e) {\textbf{05 確認 Channel}\\Developers Console
```

```
    };
\node[card,text width=40mm,left=of e] (f) {\textbf{06 設定與驗收}\\憑證、Webhook、回覆};
\draw[flow] (a)--(b);\draw[flow] (b)--(c);\draw[flow] (c)--(d);
\draw[flow] (d)--(e);\draw[flow] (e)--(f);
\node[note,below=5mm of d,text width=46mm] {Provider 確認後無法任意更換，請先核對。};
\end{tikzpicture}
\caption{帳號開通與 Channel 建立路線圖（自行繪製）}\label{fig:registration}
\end{figure}

\section{確認設定位置與權限}
在 Channel 的 Basic settings 檢查 Channel ID、名稱與 Channel secret；在 Messaging API 分頁設定 token、Webhook 及加入好友資訊。Channel ID 是識別碼，不能代替 secret 或 token。

如果找不到 Channel，先確認兩個管理平台是否使用相同登入身分、Provider 是否正確，以及帳號是否有管理權限。不要因為介面不同而重複建立多個官方帳號。本文採用官方帳號端啟用 API 的流程，避免沿用過時的直接新增 Messaging API Channel 教學。
\clearpage
\chapter{憑證取得與環境設定}
\section{Channel secret 與 access token}
Channel secret 是驗證 Webhook 的共享密鑰。進入 Channel 的 Basic settings，取得 Channel secret，保存在伺服器環境變數。收到請求後，使用它計算原始 body 的 HMAC-SHA256，再進行 Base64 編碼，與 X-Line-Signature 比對。

Channel access token 用於授權伺服器呼叫 Messaging API，放在 HTTP Authorization 標頭，格式為 Bearer 加上 token。教學環境可在 Messaging API 分頁的長效 token 區塊發行 token；若已有 token，勿為了複製而反覆重發。正式服務宜評估官方建議的可指定效期 token v2.1 與輪替策略。
\begin{table}[htbp]
\centering\caption{憑證用途與設定對照}\label{tab:credentials}
\begin{tabular}{p{35mm}p{46mm}p{52mm}}\toprule
資料 & 取得位置 & 用途\\\midrule
Channel ID & Basic settings & 識別頻道，不可作驗證密鑰\\
Channel secret & Basic settings & 驗證 Webhook 簽章\\
Access token & Messaging API／token 發行流程 & 授權回覆 API\\
Reply token & Webhook 事件 & 單次回覆特定事件\\\bottomrule
\end{tabular}
\end{table}

\begin{figure}[htbp]
\centering
\begin{tikzpicture}[node distance=7mm]
\node[card,text width=140mm,minimum height=17mm] (a) {\textbf{Channel secret｜確認事件來源}\\LINE 平台與接收端共享，用於 HMAC-SHA256 簽章驗證。};
\node[card,text width=140mm,minimum height=17mm,below=of a] (b) {\textbf{Access token｜授權呼叫 API}\\機器人伺服器持有，放在 Authorization: Bearer 標頭。};
\node[card,text width=140mm,minimum height=17mm,below=of b] (c) {\textbf{Reply token｜回覆這一次事件}\\從 Webhook 取得，僅供單次回覆，不能當成長效憑證。};
\node[note,below=4mm of c] {三者責任不同；不得互相替代，也不要放入公開截圖與版本紀錄。};
\end{tikzpicture}
\caption{三種憑證的用途與生命週期概念（自行繪製）}\label{fig:credentials}
\end{figure}
```

```
207
208 \section{PowerShell 環境變數}
209 使用 Python 3.10 以上版本。範例僅用標準函式庫，無須安裝 Flask 或 LINE SDK。請在啟動程式的同
    一個終端機設定以下變數；占位文字須替換為自己的真值。
210 \begin{lstlisting}[caption={設定環境變數並啟動服務},label={lst:start}]
211 $env:LINE_CHANNEL_SECRET = 'YOUR_CHANNEL_SECRET'
212 $env:LINE_CHANNEL_ACCESS_TOKEN = 'YOUR_CHANNEL_ACCESS_TOKEN'
213 python app.py
214 \end{lstlisting}
215 環境變數僅對目前終端機及其子程序生效。程式不會自動讀取 .env；隨附 .env.example 只說明欄位。
    缺少變數時程式會立即退出，避免誤以為空憑證也能完成回覆。
216 \section{憑證保護}
217 不要把真實憑證放入程式碼、報告、截圖或 Git commit。若曾外洩，應在平台撤銷／重發，更新服務環
    境後再測試；只刪除檔案無法移除既有 Git 歷史中的秘密。程式紀錄只保留必要錯誤訊息，不輸出
    聊天內容、Authorization 或 reply token。
218 \clearpage
219 \chapter{Webhook 設定與測試}
220 \section{建立公開 HTTPS 入口}
221 本機程式僅監聽 127.0.0.1:8000。LINE 無法連到個人電腦的 localhost，因此開發時需建立公開
    HTTPS 通道，正式服務則部署至具有有效憑證的伺服器。
222 \begin{enumerate}
223 \item 先執行程式碼 \ref{lst:start}，確認看到 Listening 訊息。
224 \item 從 \url{https://ngrok.com/docs/} 安裝 ngrok，登入自己的帳號並依官方指引設定
    authtoken。不要把 ngrok 的 token 誤當 LINE token。
225 \item 另開終端機執行 \texttt{ngrok http 8000}，保持兩個程序持續運作。
226 \item 複製 ngrok 顯示的 HTTPS forwarding 網址，後面加上 \texttt{/callback}。例如 \url{
    https://example.ngrok-free.app/callback} 是格式示意，並非可用端點。
227 \item 開啟 Developers Console 的 Channel，於 Messaging API 分頁編輯 Webhook URL，貼上完整
    網址並儲存。
228 \end{enumerate}
229 \section{Verify 與手機端驗收}
230 點選 Verify 測試連通性，再啟用 Use webhook。Verify 可能傳送 events 為空陣列的請求；程式必
    須通過簽章驗證並回覆 HTTP 200，不應因缺少訊息就拋出例外。
231
232 接著透過 Channel 顯示的 QR code 將官方帳號加入好友。依管理介面的回應設定關閉會與程式重複回
    覆的自動回應；視需求停用加入好友歡迎訊息。傳送「你好 LINE」，預期收到完全相同的文字。再
    傳送圖片，範例會略過並回覆 Webhook 200，不會產生文字回聲。
233
234 \textbf{兩種成功必須分開確認：}Verify 成功表示端點可接收驗證請求；手機實際收到回覆才證明事
    件、token 與 Reply API 全部連通。本文未代替讀者登入個人帳號，這兩項須由本人完成。
235
236 \begin{figure}[htbp]
237 \centering
238 \begin{tikzpicture}[x=1mm,y=1mm]
239 \foreach \x/\t in {0/使用者,48/LINE 平台,108/機器人伺服器}{
240  \node[card,minimum width=33mm] at (\x,0) {\t};
241  \draw[dashed,draw=border] (\x,-8)--(\x,-94);
242 }
243 \draw[flow] (0,-17)--node[above,note]{1. 傳送文字}(48,-17);
244 \draw[flow] (48,-33)--node[above,note]{2. POST 事件＋簽章}(108,-33);
245 \node[note,anchor=west,text width=32mm] at (109,-43) {先驗證簽章\\再解析 JSON};
```

```
\draw[flow] (108,-57)--node[above,note]{3. Reply API＋token}(48,-57);
\draw[flow] (48,-73)--node[above,note]{4. 顯示回覆}(0,-73);
\draw[flow,dashed] (108,-89)--node[above,note]{5. Webhook 回傳 200}(48,-89);
\end{tikzpicture}
\caption{文字事件的同步處理時序（自行繪製；步驟 3 成功後才確認接收）}\label{fig:sequence}
\end{figure}

\section{為何驗證原始 body}
簽章以傳輸時的位元組計算，空白、換行或 Unicode 編碼的差異都會改變結果。因此必須先讀取原始 bytes 並驗證，再呼叫 JSON 解析。不能先解析 JSON、重新序列化後才驗證。範例使用固定時間比較函式降低比對過程的資訊洩漏。
\clearpage
\chapter{完整範例程式與解說}
\section{設計與執行流程}
本實作的接收端與 API 呼叫均使用 Python 標準函式庫。圖 \ref{fig:flow} 對應下列函式：verify\_signature 保護事件來源；reply\_text 建立授權請求；process\_webhook 決定哪些事件需要回覆；make\_handler 將 HTTP POST 轉交核心處理函式。

每個文字事件使用自己的 replyToken。回覆 token 只能使用一次，應收到後立即使用；官方規範要求在收到 Webhook 後一分鐘內使用，重送事件另有條件，不應將 token 當成可長期保存的憑證。程式不盲目重試 API，以避免重複使用 token。

非文字事件、standby 模式或沒有可用 reply token 的事件會略過。驗證成功但 events 為空則直接回覆 200。API 網路失敗回傳 502，部署時可據此觀測錯誤。此同步教學實作未加入事件去重、佇列與資料庫，不應直接當成高流量正式服務。
\section{接收與回覆程式}
下列程式直接從交付的 app.py 匯入，左側行號由 listings 自動產生。註解採英文以方便程式碼跨環境顯示，本節以繁體中文補足解說。
程式依處理責任分成五段，行號連續對應同一份 app.py，可直接執行交付的完整檔案。
\Needspace{330pt}
\lstinputlisting[language=Python,firstline=1,lastline=20,firstnumber=1,caption={app.py：匯入模組與簽章驗證},label={lst:app}]{app.py}
\Needspace{190pt}
\lstinputlisting[language=Python,firstline=23,lastline=32,firstnumber=23,caption={app.py：建立回覆 API 請求},label={lst:reply}]{app.py}
\Needspace{600pt}
\lstinputlisting[language=Python,firstline=35,lastline=77,firstnumber=35,caption={app.py：解析事件與分類錯誤},label={lst:events}]{app.py}
\Needspace{490pt}
\lstinputlisting[language=Python,firstline=78,lastline=111,firstnumber=78,caption={app.py：HTTP 接收端與回應},label={lst:http}]{app.py}
\Needspace{300pt}
\lstinputlisting[language=Python,firstline=112,lastline=129,firstnumber=112,caption={app.py：環境檢查與服務啟動},label={lst:main}]{app.py}

\section{程式行號與逐段解說}
表中行號與程式碼 \ref{lst:app} 至 \ref{lst:main} 對照。可先閱讀核心事件處理，再追蹤驗證與回覆函式，最後確認 HTTP 接收端如何串接。
\begin{longtable}{p{17mm}p{26mm}p{105mm}}
\caption{app.py 行號與處理責任對照}\\
\toprule 行號 & 功能 & 說明\\\midrule\endfirsthead
```

```
\toprule 行號 & 功能 & 說明\\\midrule\endhead
16--22 & 簽章驗證 & 使用 secret 對原始 bytes 計算 HMAC-SHA256，再以固定時間比較驗證 Base64 簽章；此時尚未解析 JSON。\\
23--34 & 發送回覆 & 將文字與 replyToken 編為 JSON，於 Authorization 加入 access token，設定十秒逾時並呼叫 Reply API。\\
35--77 & 事件處理 & 先驗證再解析，略過非文字及 standby 事件；空事件回覆 200，API 錯誤回覆 502。\\
78--111 & HTTP 接收 & 限制路徑、內容長度及讀取時間，將原始 body 與標頭交給核心函式，再明確回傳 HTTP 狀態。\\
112--129 & 啟動與關閉 & 檢查環境變數後只監聽本機，配合 HTTPS 通道；按 Ctrl+C 時釋放伺服器資源。\\
\bottomrule
\end{longtable}
\subsection{依事件追蹤資料流}
事件進入核心函式後，先驗證來源，再解析事件陣列。每筆事件必須是文字訊息且具有 replyToken，才將文字交給 reply\_text。收到空陣列時迴圈不執行，仍回傳 200，因此能接受平台的驗證請求。
\subsection{如何根據紀錄定位問題}
API 回傳 HTTP 錯誤時，僅記錄狀態碼。例如 401 可優先檢查 access token，400 可檢查請求參數及 replyToken；連線失敗則記錄 connection failed。接收端皆回傳 502 表示上游回覆未完成，詳細分類應查看伺服器紀錄。程式不輸出 API 回應 body 或例外原文，以避免敏感資訊進入紀錄。

\section{錯誤處理與部署限制}
請求路徑限定為 /callback，輸入大小上限為 1 MiB；HTTP body 讀取與對外 API 皆設逾時。錯誤簽章及錯誤資料格式回傳 400。憑證缺漏時於啟動階段中止，避免服務啟動後才發現設定錯誤。

本範例的 HTTPServer 是單一程序的教學伺服器。正式環境應改用可靠的應用框架與伺服器、HTTPS 反向代理、祕密管理與可觀測性機制。若啟用 Webhook redelivery，須以 webhookEventId 設計持久化去重與工作狀態管理；單純把已處理 ID 存入記憶體無法承受重啟，多事件請求部分成功時也可能重複處理。須先完成這些機制再依需求啟用重送。
\clearpage
\chapter{測試、排錯與延伸應用}
\section{離線測試與線上測試的區別}
執行 \texttt{python -m unittest -v test\_app.py}，測試以假 secret 計算簽章，以 mock 取代對外傳訊，不需 LINE 憑證。涵蓋正常文字回覆、內容遭竄改、缺少簽章、空事件、非文字與 standby 事件、錯誤 JSON 結構、無效 JSON、網路錯誤及輸出請求格式。另以本機 HTTPServer 與實際 HTTP POST 驗證路徑錯誤、空 body、大小限制、無效長度、無效簽章、有效驗證請求與上游失敗，並確認錯誤紀錄只含 HTTP 狀態碼。共 17 項測試，均不使用真實憑證。

離線通過表示核心邏輯符合這些測試案例，仍須以真實帳號、有效憑證、公開 HTTPS 與手機訊息完成端對端驗收。不得以 mock 成功取代真實串接證據。
\begin{table}[htbp]
\centering\caption{驗收項目與預期結果}\label{tab:acceptance}
\begin{tabular}{p{45mm}p{85mm}}\toprule
操作 & 預期結果\\\midrule
缺少環境變數啟動 & 顯示缺少設定並退出\\
官方 Verify & 回覆 200，介面顯示成功\\
手機傳送文字 & 收到相同文字一次\\
傳送圖片 & 不產生文字回聲\\
錯誤簽章 & 400，不呼叫回覆 API\\
無效 access token & API 失敗，接收端回覆 502\\\bottomrule
\end{tabular}
```

```
\end{table}

\begin{figure}[htbp]
\centering
\begin{tikzpicture}[node distance=8mm]
\node[card,text width=140mm] (a) {\textbf{第一層：離線與本機 HTTP 測試}\\執行 17 項測試，確認簽章、事件、路徑及錯誤分類。};
\node[card,text width=140mm,below=of a] (b) {\textbf{第二層：官方 Verify}\\確認 HTTPS、/callback、secret 與 HTTP 200。};
\node[card,text width=140mm,below=of b] (c) {\textbf{第三層：手機端回覆}\\確認 Use webhook、好友關係、access token 與 Reply API。};
\draw[flow] (a)--(b);\draw[flow] (b)--(c);
\node[note,below=4mm of c,text width=140mm] {第一層已通過；第二、三層仍待真實帳號操作。若某層失敗，先排查該層的設定。};
\end{tikzpicture}
\caption{從程式邏輯到手機回覆的三層驗收（自行繪製）}\label{fig:testing}
\end{figure}

\section{線上實測與證據紀錄}
以下項目需由本人登入 LINE 平台並實際操作，尚未執行。完成後可補入去除敏感資訊的截圖，並記錄時間、操作與觀察結果。
\begin{enumerate}
\item Channel 與 Webhook 設定：擷取 API 已啟用及 Use webhook 設定畫面，遮蔽 secret、token 與不需公開的識別資料。
\item Verify 成功：確認網址含 /callback 且顯示成功，記錄測試時間。
\item 手機端文字回覆：傳送一則文字，截取使用者輸入與相同回覆，確認未發生重複回覆。
\item 非文字測試：傳送圖片，確認伺服器正常接收且未產生文字回聲。
\end{enumerate}
上述是待執行的驗收程序；隨附測試紀錄只證明離線與本機 HTTP 測試結果。
\section{常見問題排查}
\begin{longtable}{p{33mm}p{48mm}p{50mm}}
\caption{Webhook 常見錯誤與處理方式}\label{tab:debug}\\
\toprule 現象 & 可能原因 & 處理方式\\\midrule\endfirsthead
\toprule 現象 & 可能原因 & 處理方式\\\midrule\endhead
Verify 連線失敗 & 本機或 ngrok 未執行 & 確認兩個終端機持續運作\\
404 & 忘記 /callback 或路徑拼錯 & 修正完整 Webhook URL\\
400 & secret 錯誤、簽章缺漏、body 被改寫 & 核對 Channel 並保留原始 bytes\\
502 & token 過期或錯誤、網路失敗、reply token 無效 & 查平台狀態與憑證，不盲目重試\\
Verify 成功但無訊息 & Use webhook 未開、未加好友 & 啟用設定並用手機實測\\
收到兩次不同回覆 & 平台自動回應仍啟用 & 檢查官方帳號回應設定\\
隔天無法連線 & 通道網址更換 & 更新 Console 的 HTTPS 網址\\\bottomrule
\end{longtable}
\section{Push、Rich Menu 與 Flex Message}
Reply 用於回覆事件；Push 可由服務主動發送通知，但仍須遵守可發送對象、方案與用量限制。預約提醒可採 Push，切勿在沒有合法用途與適當告知的情況下大量發送。

Rich Menu 在聊天畫面提供固定入口，適合查詢、預約與客服。Flex Message 以 JSON 描述卡片等介面，適合訂單或活動資訊；可先用官方模擬工具預覽，並提供有意義的替代文字。這些功能不改變接收 Webhook 時必須驗證簽章的原則。
\clearpage
\chapter{重現、版本管理與參考資料}
```

```
\section{檔案與編譯方式}
學號資料夾包含 main.tex、app.py、test\_app.py、.env.example、README.md、編譯後 PDF 與驗證紀錄。Windows 安裝 XeLaTeX 後，在此資料夾執行下列指令三次，使頁碼與四種目錄穩定。
\begin{lstlisting}[caption={XeLaTeX 編譯命令},label={lst:compile}]
xelatex -interaction=nonstopmode -halt-on-error -jobname=412630096 main.tex
\end{lstlisting}
本文件使用 xeCJK 與標楷體（DFKai-SB）顯示繁體中文，listings 產生程式行號，TikZ 繪製架構圖，hyperref 建立目錄與交叉引用。若更換作業系統，請將字型名稱改為已安裝且可顯示繁體中文的字型後重新編譯。
\section{Git 與 GitHub 交付}
本機以功能分段提交：先建立文件架構與執行說明，再加入機器人與測試，最後完成教學排版及驗證。這些提交包含實際內容變更，不使用空提交湊數。本作業指定 GitHub 儲存庫為 Charlotte1107/professional-layout-design。完成編譯與驗證後，將本機提交推送至該儲存庫，並核對遠端分支與本機版本一致。

version-history.txt 記錄實際本機提交，repository.bundle 保存可還原的版本歷史。ZIP 不包含真實憑證；.gitignore 排除 .env 與編譯暫存檔。雲端版本可於指定儲存庫查閱；實際推送與檔案完整性結果另記錄於交付驗證紀錄。
\section{結論}
完整串接須同時完成官方帳號啟用、Channel 設定、秘密管理、HTTPS 端點、簽章驗證與訊息回覆。最容易誤判的部分是把 Verify 成功視為所有功能完成；透過離線核心測試與手機端驗收分層檢查，才能定位問題並確認真正的使用者體驗。
\section{官方參考資料}
以下資料查閱日期均為 2026 年 11 月 13 日。本文步驟以自行整理方式呈現，架構圖為自行繪製，未使用虛構的操作截圖。
\begin{enumerate}
\item LINE Developers，Get started with the Messaging API：\par\noindent\url{https://developers.line.biz/en/docs/messaging-api/getting-started/}。
\item LINE Developers，Build a bot：\par\noindent\url{https://developers.line.biz/en/docs/messaging-api/building-bot/}。
\item LINE Developers，Verify webhook signature：\par\noindent\url{https://developers.line.biz/en/docs/messaging-api/verify-webhook-signature/}。
\item LINE Developers，Messaging API reference（Reply message、Webhook event）：\par\noindent\url{https://developers.line.biz/en/reference/messaging-api/}。
\item ngrok，官方使用說明：\url{https://ngrok.com/docs/}。
\end{enumerate}
\chapter{本文件 LaTeX 原始碼}
本章使用 lstinputlisting 直接載入 main.tex，讓實際排版來源可供查核。載入的是文字內容，不會遞迴執行其中指令；標題會自動加入程式目錄。修改本文後重新編譯，即可同步更新原始碼清單。
\lstinputlisting[language={[LaTeX]TeX},caption={main.tex：本文件完整 LaTeX 原始碼},label={lst:source}]{main.tex}
\end{document}
```